

TALLER INTEGRADOR EN REACT

Construcción de portafolio profesional como aplicación web

Basado en las sesiones 1 a 3 del curso de Programación Web con React y articulado con un temario avanzado hipotético.

Elemento	Definición
Tecnología central	React con Vite
Producto final	Portafolio profesional web
Modalidad sugerida	Individual o parejas
Duración sugerida	1 a 2 semanas de trabajo autónomo
Nivel esperado	Fundamentos de React + extensión avanzada guiada

Nota para el docente: este taller puede usarse como actividad integradora después de la Sesión 3 o como puente hacia una sesión avanzada de rutas, arquitectura, despliegue y buenas prácticas de producto.

1. Enunciado del taller

Situación problema. Cada estudiante deberá construir un portafolio profesional como una aplicación web desarrollada con React. El portafolio debe presentar su perfil, habilidades, proyectos, experiencia formativa o laboral, datos de contacto y una navegación clara. La aplicación debe demostrar dominio de componentes, props, estado, eventos, renderizado de listas, formularios controlados, validaciones y consumo de datos desde una fuente externa o simulada.

Reto central. No se trata de crear una página estática de presentación personal. El producto debe comportarse como una aplicación React organizada, mantenible e interactiva, capaz de renderizar información desde estructuras de datos y, como mínimo, consumir una sección desde un archivo JSON o una API simulada.

Producto esperado. Una web de portafolio profesional lista para ser presentada, navegada y explicada técnicamente. El estudiante debe poder defender las decisiones de estructura, componentes, estado, datos y diseño.

2. Relación con las sesiones vistas

Sesión trabajada	Conceptos usados en el taller	Evidencia dentro del portafolio
Sesión 1: Introducción a React	JSX, componentes, props, composición, listas, key, estructura inicial con Vite.	Componentes reutilizables como Header, Hero, ProjectCard, SkillList, ContactSection.
Sesión 2: Interactividad	useState, eventos, formularios controlados, validación, renderizado condicional, filtros.	Formulario de contacto validado, filtros de proyectos o habilidades, mensajes condicionales.
Sesión 3: APIs	fetch, useEffect, estados loading/error/success, servicios, datos remotos o simulados.	Carga de proyectos, experiencia o testimonios desde JSON Server, archivo JSON público o mock API.

3. Temario avanzado hipotético que alimenta el taller

Aunque el taller se puede resolver con lo aprendido hasta la Sesión 3, se proponen criterios avanzados para estudiantes que quieran ir más allá o para una versión de mayor exigencia académica.

Eje avanzado	Aplicación en el portafolio
Arquitectura de carpetas	Separar components, pages, data, services, hooks, assets y styles.
React Router	Crear rutas como Inicio, Proyectos, Sobre mí y Contacto.
Hooks personalizados	Crear hooks como useProjects, useContactForm o useFilteredProjects.
Diseño responsive	Construir una experiencia adaptable a móvil, tablet y escritorio.
Accesibilidad básica	Usar etiquetas semánticas, alt en imágenes, labels en formularios y contraste adecuado.
Persistencia simulada	Consumir datos desde JSON Server, archivo JSON o API mock.
Optimización inicial	Separar componentes, evitar duplicación y mantener datos fuera del JSX cuando sea posible.
Preparación para producción	Ejecutar build, revisar errores y dejar instrucciones de instalación en README.

4. Objetivos de aprendizaje

- Construir una aplicación web con React a partir de componentes reutilizables.
- Organizar información profesional en estructuras de datos y renderizarla dinámicamente.
- Aplicar props, listas y key para mostrar colecciones de proyectos, habilidades o experiencias.
- Implementar interactividad mediante estado y eventos.
- Crear un formulario controlado con validaciones mínimas.

- Consumir datos desde una fuente externa o simulada usando fetch y useEffect.
- Manejar estados de carga, error y contenido vacío.
- Diseñar una interfaz clara, responsive y presentable como producto profesional.
- Explicar técnicamente la arquitectura del proyecto y las decisiones tomadas.

5. Requisitos funcionales obligatorios

Código	Requisito	Descripción
RF-01	Página de inicio o hero	Debe presentar nombre, rol profesional o académico, descripción breve y llamada a la acción.
RF-02	Sección Sobre mí	Debe incluir perfil, intereses, objetivo profesional y una foto o avatar opcional.
RF-03	Sección de habilidades	Debe renderizar habilidades desde un arreglo de datos, no escritas manualmente una por una.
RF-04	Sección de proyectos	Debe renderizar al menos tres proyectos usando un componente reusable ProjectCard.
RF-05	Filtro de proyectos	Debe permitir filtrar proyectos por categoría, tecnología o nivel.
RF-06	Formulario de contacto	Debe incluir nombre, correo, asunto y mensaje.
RF-07	Validaciones	Debe validar campos obligatorios y formato básico del correo.
RF-08	Renderizado condicional	Debe mostrar mensajes de error, éxito o estado vacío según corresponda.
RF-09	Datos remotos o simulados	Al menos una sección debe cargarse mediante fetch desde JSON, JSON Server o mock API.
RF-10	Estados de petición	Debe mostrar loading, error y success para la sección consumida desde datos externos.

6. Requisitos técnicos obligatorios

- **Proyecto con Vite:** la aplicación debe crearse usando React con Vite.
- **Componentes funcionales:** no se aceptará una solución escrita como un único componente gigante.
- **Props:** los componentes reutilizables deben recibir datos por props.
- **useState:** debe usarse para formulario, filtros o navegación local si aplica.
- **useEffect:** debe usarse para cargar datos externos o simulados.
- **map():** debe usarse para renderizar listas.
- **key estable:** las listas deben usar identificadores, no índices, cuando existan ids disponibles.
- **Inmutabilidad:** las actualizaciones de arreglos u objetos deben hacerse sin mutar el estado original.
- **Servicios:** las funciones fetch deben estar separadas en una carpeta services cuando haya consumo de datos.
- **README:** debe explicar instalación, ejecución, estructura y funcionalidades.

7. Estructura mínima sugerida del proyecto

La siguiente estructura es una guía. Puede modificarse si el estudiante justifica una organización equivalente o superior.

```
portfolio-react/
├── public/
│   └── data/
│       └── projects.json
```

```

├── src/
│   ├── assets/
│   ├── components/
│   │   ├── Header.jsx
│   │   ├── Hero.jsx
│   │   ├── SkillList.jsx
│   │   ├── ProjectCard.jsx
│   │   ├── ProjectFilters.jsx
│   │   ├── ContactForm.jsx
│   │   └── Footer.jsx
│   ├── services/
│   │   └── projects.service.js
│   ├── data/
│   │   └── skills.js
│   ├── App.jsx
│   ├── index.css
│   └── main.jsx
├── package.json
└── README.md

```

8. Secciones obligatorias del portafolio

Sección	Contenido mínimo	Componente sugerido
Header / navegación	Nombre, enlaces internos y acceso rápido a contacto o proyectos.	Header
Hero	Nombre, rol, breve propuesta de valor y botón hacia proyectos.	Hero
Sobre mí	Perfil profesional o académico, fortalezas e intereses.	AboutSection
Habilidades	Lista de habilidades técnicas y blandas renderizadas desde datos.	SkillList / SkillCard
Proyectos	Mínimo tres proyectos con título, descripción, tecnologías y enlace opcional.	ProjectsSection / ProjectCard
Contacto	Formulario controlado y enlaces a correo, GitHub o LinkedIn si aplica.	ContactForm
Footer	Créditos, año, nombre y enlaces secundarios.	Footer

9. Modelo de datos sugerido

Los proyectos pueden representarse con una estructura similar a la siguiente:

```

[
  {
    "id": "p1",
    "title": "AgendaFlow Lite",
    "category": "React",
    "level": "Académico",
    "description": "Aplicación para gestionar citas con componentes, estado y consumo de API.",
    "technologies": ["React", "Vite", "CSS", "JSON Server"],
    "repositoryUrl": "https://github.com/usuario/agendaflow-lite",
    "demoUrl": ""
  }
]

```

Las habilidades pueden representarse así:

```

export const skills = [
  { id: "s1", name: "React", type: "Frontend" },
  { id: "s2", name: "JavaScript", type: "Lenguaje" },
  { id: "s3", name: "Comunicación", type: "Blanda" }
];

```

10. Formulario de contacto

El formulario no necesita enviar correos reales. El objetivo académico es demostrar manejo de estado, eventos, validación y renderizado condicional.

- Campos mínimos: nombre, correo, asunto y mensaje.
- Debe validar que todos los campos obligatorios estén completos.
- Debe validar que el correo tenga una forma básica válida.
- Debe mostrar mensajes de error debajo del campo correspondiente.
- Debe mostrar un mensaje de éxito cuando el formulario pase las validaciones.
- Debe limpiar el formulario después del envío exitoso.

11. Requisitos de diseño y experiencia de usuario

- La interfaz debe ser responsive y legible en móvil y escritorio.
- Debe existir jerarquía visual: títulos, subtítulos, tarjetas, botones y secciones claramente diferenciadas.
- Debe usarse una paleta de color consistente.
- El texto debe tener contraste suficiente.
- Los botones deben tener estados visuales claros.
- La navegación debe permitir llegar fácilmente a las secciones principales.
- No se aceptarán portafolios que sean solo texto plano sin estructura visual.

12. Requisitos avanzados opcionales

Nivel	Mejora opcional	Valor académico
Intermedio	Agregar modo claro/oscuro.	Profundiza estado global o estado compartido.
Intermedio	Agregar filtros combinados por categoría y tecnología.	Refuerza lógica de arreglos y estado.
Intermedio	Agregar ordenamiento de proyectos.	Refuerza manipulación inmutable de colecciones.
Avanzado	Agregar React Router.	Introduce navegación por páginas.
Avanzado	Crear hooks personalizados.	Mejora separación de responsabilidades.
Avanzado	Consumir datos desde JSON Server.	Refuerza métodos HTTP y flujo cliente-servidor.
Avanzado	Agregar despliegue en Vercel, Netlify o GitHub Pages.	Conecta el proyecto con publicación real.
Avanzado	Agregar pruebas manuales documentadas.	Promueve validación funcional y criterio profesional.

13. Entregables

Entregable	Formato	Condición de aceptación
Código fuente	Repositorio GitHub o ZIP	Debe ejecutar con npm install y npm run dev.
README	Markdown	Debe explicar objetivo, instalación, scripts, estructura y funcionalidades.
Capturas	Imágenes dentro del README o carpeta docs	Mínimo vista móvil y escritorio.
Documento breve de defensa	PDF o Markdown	Debe explicar componentes, estado, datos, validaciones y consumo de API.
Demo opcional	URL pública	Si se despliega, debe estar enlazada en el README.

14. Condiciones mínimas para aprobar

- La aplicación debe iniciar correctamente sin errores críticos en consola.
- Debe estar hecha con React y no como HTML estático aislado.
- Debe tener al menos cinco componentes propios.
- Debe renderizar una lista usando map().
- Debe usar props en componentes reutilizables.
- Debe usar useState para manejar interacción.
- Debe tener un formulario controlado con validaciones.
- Debe consumir al menos una fuente de datos con fetch y useEffect.
- Debe mostrar estados de carga y error para los datos externos.
- Debe incluir README con instrucciones suficientes para ejecución.

15. Criterios de evaluación y rúbrica

Criterio	Peso	Excelente	Aceptable	Insuficiente
Estructura React	15%	Componentes claros, reutilizables y bien organizados.	Componentes funcionales pero con organización mejorable.	Aplicación monolítica o componentes sin responsabilidad clara.
Uso de datos y listas	15%	Datos bien modelados, listas con map y keys estables.	Listas funcionales con algunos detalles mejorables.	Datos quemados manualmente sin renderizado dinámico.
Estado e interactividad	15%	Uso correcto de useState, filtros y renderizado condicional.	Interactividad básica funcional.	No hay interacción real o el estado está mal manejado.
Formulario y validación	15%	Formulario controlado, validaciones claras y feedback visual.	Formulario funcional con validaciones mínimas.	Formulario incompleto o sin validaciones.
Consumo de datos	15%	fetch, useEffect, loading, error y servicio separado.	Consume datos, pero con poca separación o manejo parcial de errores.	No consume datos externos o simulados.
Diseño y UX	15%	Interfaz responsive, coherente y profesional.	Diseño aceptable con problemas menores.	Interfaz descuidada, difícil de usar o no responsive.
Documentación y defensa	10%	README completo y explicación técnica sólida.	README básico pero suficiente.	Sin documentación útil o no puede explicar el código.

16. Guion de defensa oral sugerido

Cada estudiante debe estar preparado para explicar su proyecto en 5 a 8 minutos siguiendo esta estructura:

1. Presentar el objetivo del portafolio y el perfil profesional que comunica.
2. Mostrar la estructura general de carpetas y componentes.
3. Explicar dos componentes reutilizables y qué props reciben.
4. Mostrar una lista renderizada con map() y explicar la key usada.
5. Explicar dónde se usa useState y para qué.
6. Mostrar el formulario de contacto, sus validaciones y mensajes.
7. Explicar qué sección consume datos externos y cómo se manejan loading y error.
8. Mostrar la interfaz en tamaño móvil o explicar cómo se trabajó el responsive.
9. Cerrar mencionando una mejora futura del portafolio.

17. Lista de chequeo antes de entregar

- El proyecto ejecuta correctamente con `npm run dev`.
- No hay errores críticos en consola.
- El portafolio tiene identidad visual propia.
- Hay mínimo tres proyectos publicados o simulados.
- Las habilidades se renderizan desde datos.
- El formulario muestra errores y éxito.
- La sección remota tiene loading y error.
- El README tiene instrucciones claras.
- El código está indentado y organizado.
- Los nombres de variables y componentes son descriptivos.

18. Recomendaciones para el estudiante

- Primero construya la versión estática por componentes; después agregue estado e interacción.
- No empiece por los estilos. Primero asegure estructura, datos y flujo.
- Use datos simulados realistas, aunque todavía no tenga experiencia laboral formal.
- No copie un template completo sin entenderlo: debe poder defender cada sección.
- Evite escribir todos los proyectos manualmente en JSX; use arreglos y componentes.
- Pruebe el formulario con campos vacíos y correos inválidos.
- Apague temporalmente la fuente de datos para verificar el estado de error.
- Revise el proyecto en móvil antes de entregar.

19. Cierre del enunciado

Este taller busca que el estudiante convierta los fundamentos vistos en clase en un producto propio, defendible y útil. El portafolio no solo evalúa conocimiento técnico de React; también evalúa criterio de presentación, organización de información, responsabilidad en la estructura del código y capacidad para explicar una solución web como producto.